DOI:10.1145/3678165

Exploring ways to include a software system as an active member of its own design team, able to reason about its own design and to synthesize better variants of its own building blocks as it encounters different deployment conditions.

BY BARRY PORTER, PENN FAULKNER RAINFORD, AND ROBERTO RODRIGUES-FILHO

Self-Designing Software

IN COMPUTING, WE have a great diversity of tools at our disposal to perform any given task. We know many different sorting algorithms, cache eviction policies, hash functions, compression algorithms, scheduling approaches, and so on. And at a higher level, we have many ways to combine these alternatives into suitable systems. Many trainees in software engineering will have learned these alternatives and have a set of heuristics they can draw on to make reasonable design choices for each new problem. Selecting the right set of tools for a new task is based on a mixture of engineering experience to narrow the initial design options, with deployment-based feedback to fine-tune (or sometimes entirely redesign) the solution. A simple example is given in Figure 1. What if our software could choose and refine the best design for a task by itself, in real time, with continuous feedback and reassessment as the nature of its task changes?

This has, broadly speaking, been the goal of the autonomic computing and self-adaptive systems research communities for the last two decades.[13,14,21,29] Over the last seven years, we have been designing a foundational approach to this challenge, in which software systems gain fundamental insights into their own design and are able to both learn alternative designs and synthesize new building blocks which are better tuned to a given context.

The systems we build draw on a large pool of potential building blocks, where each building block implements a specialized interface, like a cache with `put()` and `get()` functions, or a scheduler with `addTask()` and `getNextTask()` functions. Each such building block declares an interface it provides and may declare a set of interfaces it requires from other building blocks. This approach has roots in component-based software systems,[6] though we forego wiring diagrams or configuration files which can be cumbersome to use at scale; instead, we discover potential system designs in an entirely automated way.

The kinds of interfaces we design tend to describe a single concept, but

» key insights

- The ability to hot-swap code at runtime, safely and with generality, allows software to continuously and autonomously reason about its own design.
- When alternative implementation variants of a software component, or group of components, suit different deployment conditions, software can learn how to optimize a metric of interest by changing its design to match its environment.
- By capturing short traces of function call sequences in a running system, we can replay those traces offline to automatically synthesize higher-performance implementation variants and inject those individuals into the online design pool.

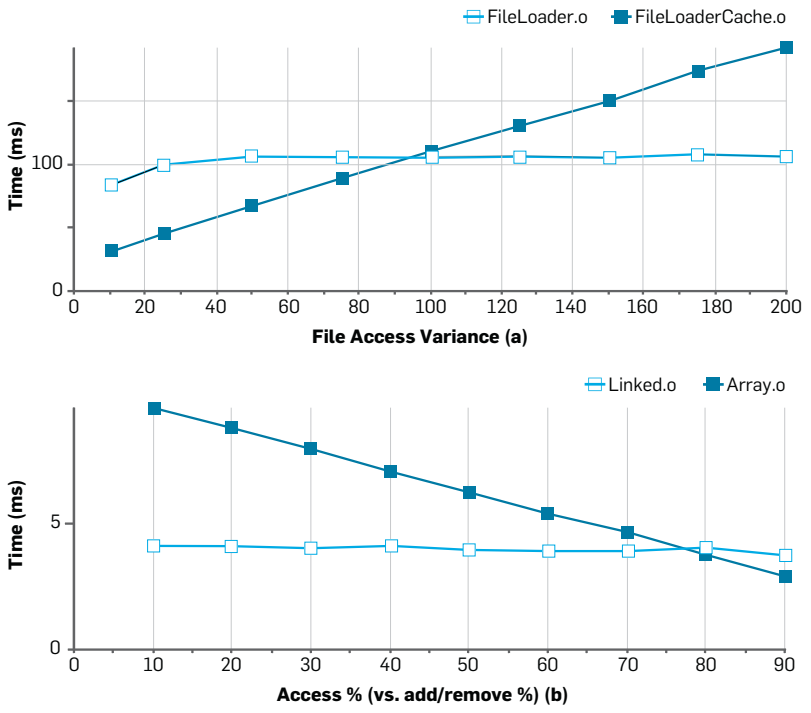
ILLUSTRATION BY BEN LEWIS GILES

Divergent Optimality of Alternative Software Implementations

In software, there are often many different ways to build the same high-level behavior, even for simple concepts.

The graph in Figure 1a shows two different implementations of a file-access API, one of which always goes straight to disk and the other which uses an in-memory cache. As we slowly increase the diversity of file requests (on the x-axis) among an increasingly larger set of files, we begin to see that the cache-assisted variant spends too much time searching the cache, gaining a miss, and falling back to disk too frequently, leading to the non-cache variant performing better in average per-call timing.

Figure 1. Two implementations of a file-access API (a) and two implementations of a list abstract data type (b).



The graph in Figure 1b shows two different implementations of a list-abstract data type: one backed by a linked list and one by an expandable array. Here, list-access behavior becomes more skewed toward index-based access of existing contents rather than additions and removals of cells. We eventually see that the array-based implementation begins to outperform the linked-list one in average per-access timing.

they are sufficiently abstract that they support multiple distinct implementations using different algorithms, internal data structures, or entirely different sub-architectures with different required interface sub-graphs. This allows us to supply a system with *variation*, which can be searched (a technique somewhat related to n-version design used in classical fault tolerance,[3] but here applied to performance objectives). Variation may include, for example, alternative search or sort algorithms, cache eviction policies, or scheduling policies, any of which may have different dependency sub-trees representing alternative sub-architectures.

The building blocks we use tend to be small, at 100–200 lines of code; when we compose many of these blocks together into a system, we then gain a potentially very large pool of combinatorial variation to explore in an automated way.

In this article, we describe the key challenges in realizing a self-designing software system, including those that are solved or partially solved, and those that are open research avenues. Along the way, we discuss what it might mean to be a meta-engineer of the future, populating variation pools, determining metrics of *self* and *environment*, guiding the direction of learning, and contributing human creativity when automated approaches reach a dead end.

Adaptation with Assurance

Our core approach uses a large pool of small building blocks, each with specialized interfaces, and injects implementation variation into a subset of those interfaces, such as different sorting algorithms or cache eviction policies. Each such implementation variant performs the same high-level task, and would pass the same unit tests for verification, but achieves that task in a different way. At runtime, we can then automate a search over the various possible compositions of these building blocks alongside an objective function (such as average responsiveness of the system).

When we switch from one available composition of building blocks to another, we perform one or more *hot swaps of code within the running system*. A hot swap entails loading the alternative implementation block from disk into program memory, performing any necessary state transfer between the existing implementation and the new one, and then unloading the previous implementation from memory. This is a bit like changing an aircraft engine for an alternative design while in flight, depending on the atmospheric conditions around the aircraft. As we are performing self-design on live software deployed in production environments, these hot swaps need to be fast and seamless, such that users notice no apparent disruption to the system's service. In some cases, we perform tens of such hot swaps in less than a second where the measurement fidelity of an objective function supports a sufficiently rapid search.

Performing hot swaps of code in a live system is fraught with risk, however, as mainstream programming languages are not designed with the necessary soundness principles to

offer assurances of correct behavior (compared, for example, with the extensive support for soundness via type safety[9,19]). Projects that do hot swap code in live systems therefore tend to use a very strict set of programming conventions under carefully scoped conditions in an attempt to mitigate the worst pitfalls.

Such delicate and tightly scoped programming is at odds with the scale and complexity of modern software; we therefore need novel solutions to support generalized self-designing software for full-stack systems. Our approach has been to analyze why hot-swapping code is risky and to design a new systems programming language, complete with rich, object-oriented design patterns and multi-threaded functionality, which is founded on hot-swap soundness. The rules implied by the grammar of our language make it very difficult to write code which is *not* hot-swap safe.

The key challenge is illustrated in Figure 2. Imagine we have two different compositions of a system, A and B, which both do the same high-level thing but are composed of different building blocks. Each individual building block has an internal state machine encoded in its program text; each composition has a combined state machine of all the individual building blocks put together. When verifying the correctness of A and B, it is reasonable to use the same set of unit tests for both system variants (since they apparently do the same thing). It is also reasonable to test both system variants *in isolation*: We start up composition A, run our unit tests, then shut it down. We then start up composition B, run our unit tests again, and shut B down.[a] Because verification is necessarily done in isolation on each composition, it is then up to the programming language to ensure that switching *between* compositions, at any arbitrary point in time, cannot go wrong. The reasons a hot swap might go wrong can be summarized as follows: Given that compositions A and B have at least some variation in their composite state machines, switching between A and B can easily result in a *hybrid* of the two separate state machines and can land the system in a particular state which exists in *neither* of the composite state machines of A or B. Such a state cannot have been verified, under the above verification assumptions, and therefore has undefined behavior.

The goal of a programming language which offers assurance of hot-swap safety is simply to ensure that such hybrid states are impossible (or at least very difficult, without deliberate effort from the programmer to try to create such a state). Our solution is the Dana adaptive programming language,[20] which treads the seemingly narrow path of offering such assurance while also providing a rich object-oriented and concurrent programming environment. Dana has been a mature platform for several years, with a standard library offering everything from low-level network abstractions up to graphic user interface (GUI) toolkits, all designed as collections of strongly separated building blocks.

We also use Dana to write programs we call *meta-composers*, which are our agents of automated self-design: They locate potential compositions of building blocks, inject measurement probes into systems to track an objective function (such as performance or energy consumption), and track features of the deployment environment to correlate context against objective functions.

Learning What Is Best

Given a technology that can assure generalized safety of arbitrary code hot swaps in live systems, the next challenge is to build a control system to orchestrate the design of those systems as a continuous process in real time. We begin with a simple abstraction,

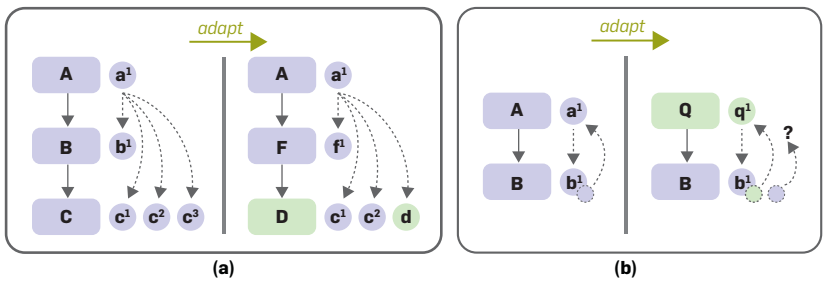
Safety Challenges in Hot-Swapping Code

Hot swaps of code in a live system present challenges when we yield a mixture of two different state machines, ending up in a particular state that exists in neither the set of reachable states of the prior composition nor the new one.

Consider the simple example in Figure 2a, in which we are hot-swapping a single building block. Building blocks can be considered analogous to classes for our purposes here, with this example showing a factory pattern where an instance of class A generates products from the factory class B. The left side shows the original system, with the post-hot-swap one on the right.

Having hot-swapped the factory, on the right we have a resulting state with a mixture of products from two different factories; while it may work, this state exists in neither isolated state machine and is therefore outside of verifiable behavior.

Figure 2. Hot-swapping a single building block (a) and a typical observer pattern relationship (b).



Our second example, in Figure 2b, illustrates a typical observer pattern relationship. Object *a*′ has registered for callbacks from object *b*′, but class A is later hot-swapped to class Q. Object *b*′ is left with a hanging reference to object *a*′, which is no longer in-use in the system.

Both examples are deliberately simple, but they are also easily done in normal use of a contemporary programming language, without any intended harm from an engineer. More complex versions of this problem, which may span across the interactions of many objects, are much more difficult to trace.

a It is not reasonable, by contrast, to expect tests in which composition A is run for a while, with some tests executed, then a switch to composition B for a while, so that more tests can be executed. This is because we have no sense of *when* hot swaps will occur at runtime, and testing every possible moment is an infinite space.

in which we automatically discover potential compositions of a system and give each entire composition a simple label. A meta-composer program examines the entry-point building block of a target system (the building block with the main method). It then examines each of the required interfaces of this building block, and for each one the local system is searched for all building blocks that provide an interface of matching name and type. Each of *those* building blocks is examined for their own required interfaces, recursively constructing a permutation graph. The result of this automated search is a set of entire compositions of the target system; each composition is then given a simple, unique text label.

We can then ask a meta-composer program to change to a labeled composition, at which point it will perform a diff between the currently selected composition and the requested one and will calculate a *sequence* of hot swaps of individual building blocks to reach that requested composition. This simple abstraction allows us to connect compositional search to optimization processes such as reinforcement learning, which can explore labeled actions to understand their apparent reward. While optimization algorithms can thus operate at the level of labeled actions and rewards, in practice they are making complex choices on the design of a live software system. In our work, we assume optimization is performed almost entirely online, with no or very limited pre-modeling, such that we maximize the ability of a system to deal with real uncertainty in its operations. Relying on this assumption, even if we do later add pre-modeled scenarios, means we tend to design our optimization processes to be able to operate well with significant uncertainty in deployment as and when it occurs.

We have applied this simple abstraction across a range of application domains, including the design of Web servers,[22,27] caching systems, distributed workload scheduling,[7] microservice optimization in conjunction with autoscalers,[8] and the automated distribution of program elements over a network for interactive media streaming.[4] From across these experiences, we have developed a common process that we, acting as meta-engineers, consistently follow when taking on a new problem domain.

We begin by building out the primary system from a set of new, tightly scoped building blocks, such as an HTTP stream processor for a Web server or a media track renderer for a media-streaming application. These building blocks will reference a wide range of existing APIs in the Dana standard library, such as abstract data types, caches, or common data-processing algorithms, many of which have existing implementation variation available.

We then determine our objective function for the system (which may change in-deployment). This may be formed from a single metric (such as throughput) or may be composed of several different metrics (such as energy consumption, monetary cost, or measurements of user-experience quality, like rendering fidelity or system responsiveness). We then decide how to capture these metrics in the live system. Some may be captured by standard probes, such as power monitors; others, such as throughout, may inject a standard timing probe in a particular location of the system (to capture the execution time of one or more function calls across a particular interface); or some may simply be captured as the existence of a particular implementation variant (such as a renderer) within a chosen composition known to deliver a particular quality of experience. The particular weightings of a composite objective function can be tuned at design time and later in deployment.

We lastly determine how to *featurize* the deployment environment using a second set of measurement probes. This step is optional, but it provides a working memory to an optimization process, which can lead to quicker decisions when deployment environment conditions recur. Features may be simple binary ones, such as whether a host computer is currently charging or discharging its battery, or may be computed features from primary raw data, such as sampling a Web server's incoming requests to determine the current mixture of MIME types from a set of discrete categories, with a potential secondary analysis stage predicting how cacheable or compressible that mixture is. Given such a feature set, a classifier can then be used to continuously detect a set of *contexts* for optimization; for each newly detected context, an optimization or learning algorithm can determine which combination of building blocks gains the highest performance, and then store that knowledge alongside the classified context. This knowledge can later be retrieved quickly if that same deployment context reappears in the future.

Once a system is deployed, the way in which it is optimizing itself can be observed, considering apparent quality and velocity of its decision making, and the kinds of environments the system is experiencing can be analyzed (and the effectiveness with which those environments are being classified). At this stage, engineers may wish to develop additional implementation variation for existing APIs in their system, deriving new variants which may be better suited to some of the observed deployment environment dynamics. These can be dropped into the live system for inclusion in an expanded set of potential compositions, with engineers then able to understand how often those new variants are used. Likewise, objective functions and classification methods may be updated in real time to tune how the system is choosing to design itself under the range of conditions it is experiencing.

To help exemplify the above process, we highlight two different application domains to which we have applied it. One of the first systems we designed with this process was a simple Web server. Here we wanted to understand the level of useful variation available—and our ability to automatically detect that variation against different deployment conditions—in a relatively simple system which has very broad use in the world. Following our meta-engineering process, we developed some of the bespoke building blocks of a Web server, including HTTP stream processors and concurrency management modules, and added variation by developing stream processors that do and do not use caching or response-compression, and those that use either or both of these features. We then added further variation to the caching and compression building blocks, with different cache eviction policies and

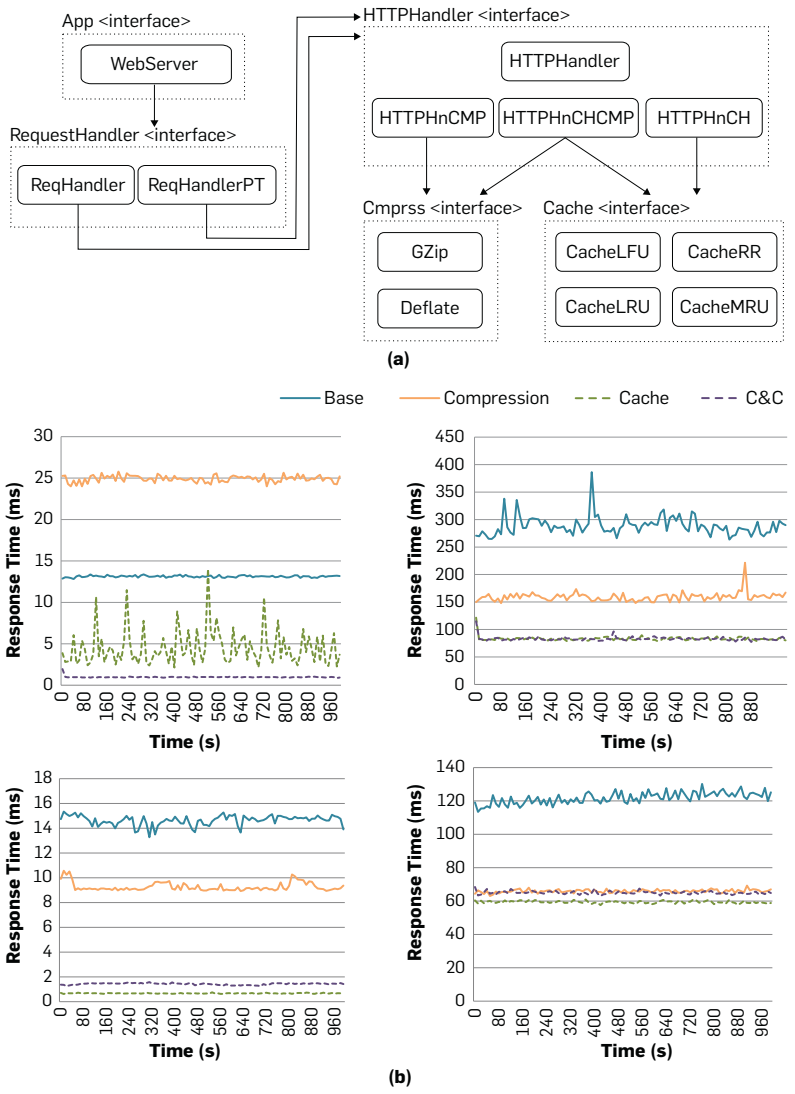
compression algorithms. This yielded 42 distinct compositions of a Web server, each uniquely labeled as described earlier. Our objective function was a simple throughput metric, determined as the execution time of a particular interface function, and our classifier detected the likely cache-ability and compress-ability range of the current request pattern. We then attached a Bayesian multi-armed bandit to the set of available software composition actions and used regression-based analysis over the building-block features of each composition to speed up the live learning process (using a linear-regression model in this case). Using regression in this way allows reasonable inferences to be made about the *likely* performance of as-yet-untried composition actions, based on similarities in their building-block sets to other composition actions that *have* been tried; this reduces the overall number of actions that need to be tried before discovering the ideal choice. Some of the results are summarized in Figure 3. From this system, we learned there is useful variation even in such a simple example, that the ideal design can be detected and chosen quickly, and that an ideal design can be learned even in realistic workloads which have relatively noisy properties. When trialing this system across varying hardware (datacenter servers versus Raspberry Pi devices), we also saw that the system would locate different compositions (designs) for the same workloads, depending on how the hardware handles different instruction and I/O mixtures. We also learned, however, that crafting a suitable classifier was a relatively manual process which required considerable knowledge of the application domain—and careful thought around how to go from sparsely sampled live metrics to useful discrete classes. We discuss this challenge later in this article.

One of our most recent application domains is in interactive media streaming, using the object-based media paradigm. In this approach to media delivery, a media experience is divided into its constituent parts, such as actors; presenters; backgrounds; synthetic digital assets, including 3D meshes or digital humans; camera angles; chapters; and information over-

Self-Design in a Simple Web Server

One of our first domains for self-designing software was a simple Web server. The main variation points are shown in Figure 3a. Featured are two different concurrency models and four stream-handler variants, with those stream handlers having different dependency sub-trees with caching and/or compression modules.

Figure 3. Main variation points in a simple Web server (a) and four main groups of features of a Web server (b).



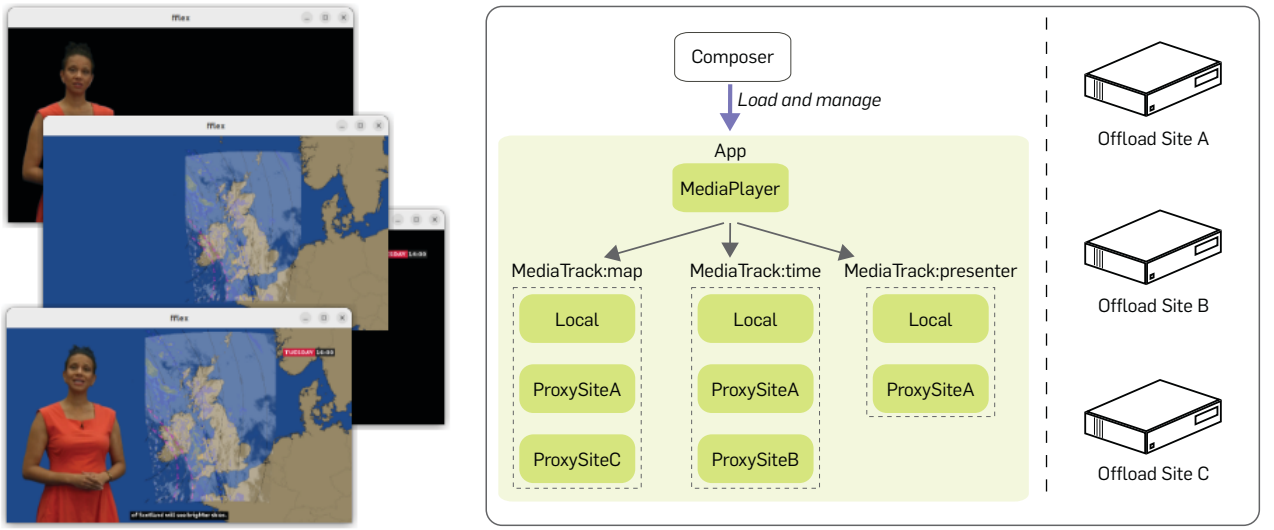
If we identify four main groups of features (Figure 3b), we see that these feature groups are likely to offer divergent performance levels under different workloads, and on different hardware. The left two graphs below are measurements from a datacenter server, while the right two graphs are the same system running on a Raspberry Pi. The top two graphs are an image-heavy workload, and the bottom two graphs are a text-heavy workload with higher variation. Across these varying conditions, we see different ideal software designs giving the best throughput.

Using a Bayesian learning algorithm, along with software feature regression to help understand the marginal contribution of each individual building block to system performance (and sharing of those expected contributions between design alternatives), we find that real-life workloads may be learned without necessarily testing all design options. Depending on the workload, a given environment may need to test between 6 and 40 of the available 42 designs to gain a clear understanding of the ideal choice.[22]

Self-Design for Compute Offloading

Object-based media is divided into separated media assets which can be dynamically composed in user-configured ways. Figure 4a shows a simple example of a weather forecast, in which the presenter, map, and time-of-forecast are delivered separately and could be configured to blend different map fidelities and styles, vocal or sign-language presenters, or levels of presentation detail. Because each element is separated, they are rendered separately and composed at playback time.

Figure 4. An example of a simple weather forecast (a), and a simplified software architecture for media playback (b).



Our (simplified) software architecture for media playback (Figure 4b) has one renderer for each kind of content. By default, these renderers reside locally, but we can inject network proxies to move selected render tasks to compute offload sites (at the edge or in the cloud) for more render-intensive content. Each offload site for each render block is injected as an implementation variant for the relevant interface, retaining our simple labeled composition approach. This simple example yields 18 distinct possible local/distributed software designs.

lays, with each viewer able to configure an experience to suit their interests. Because the configuration of the media's final presentation is user-defined, each frame of an experience is rendered at least partly in real time. Again, following our meta-engineering process, we first developed some of the bespoke building blocks of this system, including media configuration parsing, rendering loops, and bespoke renderers for each kind of content that can form an experience (the latter in collaboration with a media-streaming organization on a large research project). Our core media player is a local-only system which renders everything on the device. We next injected variation using transparent network proxies which can offload any render task to a remote computer and stream the results back to the viewer's device. We then derived renderers of different fidelity (and corresponding compute intensity) to provide quality/compute trade-offs. Again, each distinct design of the system (including those which offload various mixtures of render tasks) is simply realized as a labeled composition, such that making composition choices here drives both the local system design and its distributed architecture. As our objective function we use the average time to render each frame of the media experience, combined with the render quality achieved. In the future, we are likely to include cost and energy usage to the media-streaming organization as further factors. Our approach to distributed design, following our compositional abstraction, is summarized in Figure 4. In this application domain, we are not just seeking to learn what is locally best for each user, but also to balance the global resource landscape across the set of offload sites; if, for example, we can cache partial renders at offload sites, we may also wish to co-locate offloaded client render tasks at correlated cache sites. This suggests that multi-agent or hierarchical coordinated optimization approaches may be needed (for example, Mellodge et al.[16] and Weyns et al.[33]

Although we have made good progress in refining some of the theory and technology of a meta-engineering process, a range of hard challenges remains. We summarize some of the most pressing, which are uncertainty, generalized classification, search-space management, and synthesis of useful variation.

One of the most surprising challenges has been finding methods to deal with real *uncertainty* about a system's deployment environment conditions, in scenarios where learning is performed entirely online in the live system, with no prior modeling. Ideally, we would like to be able to deploy a system, with our variation points and our objective function instrumentation, with a zero-configuration RL algorithm to determine ideal system designs in real time. This is because if we had any certainty about the likely deployment conditions ahead of time,

we may not need to delegate design or co-design to an automated system. Every parameter we must select or tune in a RL algorithm, however, equates to making more assumptions about the likely characteristics of a deployment environment, and correspondingly reducing our openness to uncertainty about those conditions. The need to have prior certainty through well-tuned optimization or learning algorithms, many of which currently need careful parameterization to work effectively, therefore runs against the history of systems development and operations, which is littered with unexpected effects and system characteristics caused by poor assumptions about a deployment environment.[1,5,32] The AutoML research community has examined methods to automated parameter tuning of learning algorithms,[10,12] but the majority of this research lies in algorithms with distinct training and deployment phases rather than continuous online RL. Recent related research for reinforcement learning (AutoRL) offers promising potential directions which may begin to allow wider degrees of uncertainty prior to deployment.[17]

Second, we find ourselves crafting highly bespoke classification algorithms to extract useful features from the environment which correlate to useful distinct contexts to learn ideal designs for. A system which classifies its environment poorly will either miss important contexts entirely (causing unexplained variance in observed rewards) or will over-classify irrelevant contexts which make no difference to the performance of the target system—and cause needless re-evaluation of learned information for each additional context. The ability to automate the derivation of suitable domain-specific classifiers would be useful, to determine likely feature combinations that matter to a given system; alternatively, a version of a typical data-science process adapted for software engineering (that is, exploring a suite of tools such as principal component analysis or autoencoders to gain a best fit) may allow human operators to more easily craft high-utility classifiers for each system.

Third, the use of fine-grained building blocks, which have a pool of variation, can combinatorially lead to potentially large search spaces in complex systems. If learning of the best composition is performed entirely online, this can lead to long convergence times on a good solution. While such a convergence time is not necessarily disruptive, it can present periods of poor quality-of-experience when in progress. Techniques such as software feature regression (discussed above) can aid in reducing this search space, but this may only go so far. There are at least two potential, complementary solutions to this. One solution is to improve automated premodeling of performance for deployment environment conditions likely to occur by using similar approaches to automated bug finding,[15,31] avoiding long from-scratch search periods when a new context is detected. The other solution is periodic active management of the search volume by human operators who can identify and prune compositional options which appear to never be useful (potentially backed by derivation of the contexts in which they would be useful, to evidence the fortitude of those pruning decisions).

Lastly, we have learned through experience that developing useful implementation variations for actual deployment conditions can be challenging, which led us to explore automated approaches to deriving new variants. We introduce our work in this area next.

Making New Things

Our approach relies on the availability of implementation variation in the pool of building blocks used to form a given system. While there is always a degree of built-in variation below standard library APIs, creating new variations can be challenging because it involves predicting the kinds of variation likely to be useful in a given set of deployment characteristics.

Given this challenge, we investigate the potential to *automatically* synthesize useful variants from existing implementations. The most prominent approach to this problem is genetic improvement (GI),[18] where a starting source-code individual forms a population, the individuals of which then undergo successive mutation and crossover across generations. GI is typically used to fix bugs in large codebases, using the phenomena that code fragments needed to correct a bug tend to appear elsewhere in that codebase.[2]

In our case, this general approach is attractive because we can temporarily inject fine-grained measurement probes atop the interface of any building block, recording a call trace of every function call made across that interface along with the parameter values and return values of each call. We can then replay this trace offline in a GI process in an attempt to yield a better implementation for this sequence of calls, according to some objective function. The new candidate variant is injected into the selection pool of the live system, which can learn when that new variant is or is not useful.[23] The basic process of GI in this context is illustrated in Figure 5.

Unlike GI for bug fixing, here we are operating on small, self-contained units of logic, and we are seeking improvements on a scale rather than binary fixes to tests. This reduces our ability to rely on useful code being found elsewhere. Instead, we mix brand new synthesis of source code for injection alongside traditional GI mutation and crossover procedures. New code synthesis works by selecting a line of code at random, and then selecting at random from among all possible legal code insertions at that line (from variable declarations to IF-statements and FOR-loops). However, the inclusion of code-synthesis-mutations leads to a search-space size problem: In the absence of any other constraints, such as a maximum number of lines of code, we can travel infinitely far, in any direction, from our starting point. Combined with this are the typical challenges of fitness (reward) landscapes in GI: These landscapes tend to be jagged, with successive peaks and valleys, and have large neutral areas where successive mutations may demonstrate no effect to the fitness of the individual, yet a sufficient combination of such mutations may yield a positive result.[25,30] Navigating such a fitness landscape, without getting stuck in zero-utility areas, can be very challenging.

One approach to rationalize this search problem is the use of phylogenetics. In evolutionary biology, phylogenetics is the study of when different species appeared and looks for correlations between evolution and extinctions of species and geological events to infer what contributed to the evolution of particular traits.[11] While biologi-

Finding Better Alternatives with Genetic Improvement

Genetic improvement for software works by selecting a starting individual, such as a hash function, and creating a population using copies of that individual. This population forms the first generation of a GI process. Crossover is then applied to the members of that generation, sharing genetic material between individuals, followed by mutation on each individual. Mutations are performed by adjusting operators, changing operands, or synthesizing new lines of code with randomly chosen operators and operands. This overall process is illustrated in Figure 5a.

Figure 5. Genetic improvement for software (a), and a phylogenetic tree from a hash table evolution GI process (b).

We use phylogenetic analysis to help scope a GI search for good individuals and to identify highly evolvable individuals as potential starting points for future GI processes. A phylogenetic tree from a hash table evolution GI process is shown in Figure 5b.

From this tree we could extract the individual which yields the two main left and right sub-trees, or an individual from the root of the left sub-tree which eventually leads to the best individual. Both potential individuals have significant redundant material but led to high-utility future individuals. GI combined with phylogenetic analysis has allowed us to gain up to 20% improvement in new implementation variants.[23]

cally a full phylogenetic tree is inferred from fossil records and ancient DNA, in computational GI we can extract an exact history. We have used phylogenetic analysis in GI to help understand the relationship between types of mutations and fitness changes, and which kinds of genetic crossover produced the most useful results.[24] This, in turn, can be used to affect an in-progress GI run to move the search in potentially more fruitful directions by adjusting mutation weightings and crossover policies.

We have also deployed phylogenetic analysis to help reason about the *evolvability* of particular individuals, which is of particular interest in the context of our self-designing software work. The best individual from a GI process is that which has the best raw performance, but the most *evolvable* individual is that which led to the most interesting branches of the evolutionary tree, including the branch containing the eventual best individual. Rather than taking only the best individual and throwing away the rest, keeping the highest-evolvable ancestor individual A_E can contribute to a library of diverse genetic material. These A_E individuals are likely to have a range of 'junk' material in their genome (that is, source code), but this material has been useful to lead toward high-utility individuals following future mutations and crossovers. Keeping a bank of such individuals may serve as better starting points, with more evolvable genetic material, for future GI processes in newly detected environment conditions in a live software system.

Our GI work to date has been applied to example building blocks which share particular properties: They have a relationship between code changes and fitness-function effects that is more likely to show at least some correlation, and it is difficult to completely break their functionality. Examples are the hash function used by a hash table, the selection algorithm used by a scheduler, or the eviction policy used by a cache. In each example, a worst-case algorithm can always choose the same value (for example, always-return-zero) and an individual may be considered functionality-correct even if it performs very poorly against an objective function. Other functions, such as sorting, are easier to 'break' and have very different implementations. Finding a GI path between two different sorting algorithms (such as insertion sort and merge sort) is clearly a very different prospect.

To support GI navigation through more complex spaces, with potentially large areas of neutral drift in which both fitness changes are non-existent *and* a program may be in a non-working state, we need more advanced methods of search assistance. We are currently exploring tightly coupled hu-

man involvement in GI processes to achieve this, in a similar fashion to our meta-engineering process for self-designing software. While we still aim for a core GI process to run autonomously, we are examining ways to visualize the nature of the search in real time, such that a domain engineer can guide search parameters to areas they identify as under-explored or high value. This may be done via high-level nudges to mutation weightings, or injection of loosely sketched individuals representing a particular search area to explore.

Recent advances in LLMs may also fill the role we have used GI for, by starting from an existing individual and attempting to find increasingly improved ones according to some metric of interest. Recent work by Romera-Paredes et al.[28] suggests that at least some problems can be navigated by a source code-trained LLM as successive code versions, yielding potentially novel solutions. Future LLM approaches that can focus on context-specific performance improvements to existing code may work as a complementary search technique to GI, with the potential to generate incrementally better solutions at high speeds (following suitable training phases).

Summary and Outlook

The growth in software complexity has seen a wide range of mitigation strategies, including layered abstraction, separation of development from operations teams (the common DevOps model), and more recently with AI-enhanced operational support to reduce the frequency of human operator interruptions when things go wrong.

We explore a complementary path of including a software system as an active member of its own design team, able to reason about its own design and to synthesize better variants of its own building blocks as it encounters different deployment conditions.

This approach suggests the role of meta-engineers who observe the high-level measurements of decision-making processes of a system, and guide that process using human inference and creativity which goes beyond what automated approaches are capable of.

Acknowledgments

This research was partly supported by the Leverhulme Trust Research Grant 'Genetic Improvement for Emergent Software', RPG-2022-109, and by the UKRI EPSRC BBC Prosperity Partnership AI4ME: Future Personalised Object-Based Media Experiences, EP/V038087. We also thank the many collaborations through Ph.D. students, post-doctoral researchers, and academics who have contributed over time to this work, including Matthew Grieves, Christopher McGowan, Paul Dean, Zsolt Nemeth, Benjamin Craine, Alexander Wild, David Leslie, Simon Dobson, Alan Dearle, and Gordon Blair.

References

1. Ardelean, D., Diwan, A., and Erdman, C. Performance analysis of cloud applications. In *Proceedings of the 15th USENIX Symp. Networked Systems Design and Implementation*, USENIX Assoc. (2018), 405–417.
2. Barr, E. et al. The plastic surgery hypothesis. In *Proceedings of the 22nd ACM SIGSOFT Intern. Symp. Foundations of Software Engineering* (2014), 306–317.
3. Baudry, B. and Monperrus, M. The multiple facets of software diversity: Recent developments in year 2000 and beyond. *ACM Computing Surveys 48*, 1, Article 16, (Sept. 2015).
4. Carter, J. et al. Universal access for object-based media experiences. In *Proceedings of the 11th ACM Multimedia Systems Conf.* (2020), 382–385; 10.1145/3339825.3393590
5. Chow, M. et al. The mystery machine: End-to-end performance analysis of large-scale Internet services. In *Proceedings of the 11th USENIX Symp. on Operating Systems Design and Implementation*, USENIX Assoc. (2014), 217–231.
6. Crnkovic, I., Sentilles, S., Vulgarakis, A., and Chaudron, M. A classification framework for software component models. *IEEE Transactions on Software Engineering 37*, 5 (Sept. 2011), 593–615.
7. Dean, P. and Porter, B. The design space of emergent schedulingfor distributed execution frameworks. In *Proceedings of the 16th Intern. Symp. on Software Engineering for Adaptive and Self-Managing Systems*, IEEE (2021).
8. Filho, R., Pereira de Sá, M., Porter, B., and Costa, F. Towards emergent microservices for client-tailored design. In *Proceedings of the 19th Workshop on Adaptive and Reflexive Middleware*, ACM, Article 2 (2018).
9. Harper, R. *Practical Foundations for Programming Languages (2nd ed.)*. Cambridge University Press, USA (2016).
10. He, X., Zhao, K., and Chu, X. AutoML: A survey of the state-of-the-art. In *Proceedings of Knowledge-Based Systems 212* (2021), 106622; 10.1016/j.knosys.2020.106622
11. Hennig, W. Phylogenetic systematics. *Annual Rev. Entomology 10*, 1 (Jan. 1965), 97–116.
12. Kanti Karmaker, S. et al. AutoML to date and beyond: Challenges and opportunities. In *Proceedings of ACM Computing Surveys 54*, 8, Article 175 (Oct. 2021); 10.1145/3470918
13. Kephart, J.O. and Chess, D.M. The Vision of Autonomic Computing. *Computer 36*, 1 (Jan. 2003), 41–50.
14. Lalanda, P., McCann, J., and Diaconescu, A. *Autonomic Computing: Principles, Design and Implementation*. Springer Publishing Company, Inc., (2013).
15. Mao, K., Harman, M., and Jia, Y. Sapienz: Multi-objective automated testing for Android applications. In *Proceedings of the 25th Intern. Symp. on Software Testing and Analysis*, ACM, (2016), 94–105; 10.1145/2931037.2931054
16. Mellodge, P., Diaconescu, A., and Di Felice, L. Timing configurations affect the macro-properties of multi-scale feedback systems. In *Proceedings of the 2021 IEEE Intern. Conf. on Autonomic Computing and Self-Organizing Systems*, 100–109.
17. Parker-Holder, J. et al. Automated reinforcement learning (AutoRL): A survey and open problems. *J. Artificial Intelligence Research 74*, (Sept. 2022); 10.1613/jair.1.13596
18. Petke, J. et al. Genetic improvement of software: A comprehensive survey. *IEEE Transactions on Evolutionary Computing 22*, 3 (June 2018), 415–432.
19. Pierce, B. *Types and Programming Languages (1st ed.)*. The MIT Press (2002).
20. Porter, B. and Rodrigues-Filho, R. A programming language for sound self-adaptive systems. In *Proceedings of the 2021 IEEE Intern. Conf. on Autonomic Computing and Self-Organizing Systems*, 145–150; 10.1109/ACSOS52086.2021.00036
21. Porter, B., Filho, R., and Dean, P. A survey of methodology in self-adaptive systems research. In *Proceedings of the 2020 IEEE Intern. Conf. on Autonomic Computing and Self-Organizing Systems*, 168-177.
22. Porter, B., Grieves, M., Rodrigues-Filho, R., and Leslie, D. REX: A development platform and online learning approach for runtime emergent software systems. In *Proceedings of the Symp. on Operating Systems Design and Implementation*, USENIX (2016), 333–348.
23. Rainford, P. and Porter, B. Lineage selection in mixed populations for genetic improvement. In *Proceedings of ALIFE 2022: The 2022 Conf. on Artificial Life*, International Society for Artificial Life.
24. Rainford, P. and Porter, B. Using phylogenetic analysis to enhance genetic improvement. In*Proceedings of the Genetic and Evolutionary Computation Conf.*, ACM (2022), 849–857.
25. Renzullo, J., Weimer, W., and Forrest, S. Evolving software: Combining online learning with mutation-based stochastic search. *ACM Transactions on Evolutionary Learning and Optimization* (May 2023).
26. Rivera, L. et al. Using dynamic knowledge hypergraphs toward proactive AIOps through digital twins. In *Proceedings of the 32nd Annual Intern. Conf. on Computer Science and Software Engineering*, IBMCorp. (2022), 163–168.
27. Rodrigues-Filho, R. and Porter, B.H. Self-distributing systems for data centers. *Future Generation Computer Systems, 132*, Elsevier, (2022), 80–92.
28. Romera-Paredes, B. et al. Mathematical discoveries from program search with large language models. *Nature 625*, 7995 (Jan. 1, 2024), 468–475; 10.1038/s41586-023-06924-6
29. Salehie, M. and Tahvildari, L. Self-adaptive software: Landscape and research challenges. *ACM Transactions on Autonomous and Adaptive Systems 4*, 2 (2009), 14.
30. Schulte, E. et al. Software mutational robustness. In *Proceedings of Genetic Programming and Evolvable Machines 15*, 3 (Sept. 2014), 281–312.
31. Tufano, R. et al. Using reinforcement learning for load testing of video games. In *Proceedings of the 44th Intern. Conf. on Software Engineering*, ACM (2022), 2303–2314; 10.1145/3510003.3510625
32. Veeraraghavan, K. et al. Kraken: Leveraging live traffic tests to identify and resolve resource utilization bottlenecks in large scale Web services. In *Proceedings of the Symp. on Operating Systems Design and Implementation*, USENIX (2016), 635–651.
33. Weyns, D. et al. *On Patterns for Decentralized Control in Self-Adaptive Systems*. Springer, Berlin Heidelberg (2013), 76–107.

Barry Porter is professor of adaptive systems at Lancaster University, U.K.

Penn Faulkner Rainford is an associate researcher at the University of York, U.K.

Roberto Rodrigues Filho is an assistant professor at the Federal University of Santa Catarina, Brazil.

Watch the authors discuss this work in the exclusive *Communications* video. https://cacm.acm.org/videos/self-designing-software